

DOCUMENTO DE PLANIFICACIÓN · V1.0 · JUNIO 2026

Elfie

PRTS Desktop Edition

Evolución de PRTS a una aplicación de escritorio nativa con control del sistema operativo, procesamiento de IA local y personalidad configurable. Desarrollada para uso exclusivo de Sylft, con base de datos compartida con la versión web.

PROPIETARIO: SYLFT IA: ELFIE PLATAFORMA: WINDOWS 11 HARDWARE: RTX 5060
· 16 GB RAM · RYZEN 5 5600G

Contenido

1 · Visión del Proyecto	3
Estado actual de PRTS web	3
Qué cambia con Elfie Desktop	3
2 · Decisiones de Arquitectura	4
Framework de escritorio: Tauri 2.x	4
Diagrama de sistema completo	4
Hardware disponible y uso	4
3 · Módulos del Sistema	5
Control del sistema operativo	5
Monitor de recursos en tiempo real	5
Dashboard sincronizado con PRTS web	5
Personalidad de Elfie	5
4 · Capa de Voz	6
STT local (Whisper.cpp)	6
Opciones de TTS locales	6
Opciones de TTS en la nube	7
Stack de voz recomendado	7
5 · IA Local (RTX 5060)	8
Ollama + modelos recomendados	8
Estrategia híbrida local + nube	8
6 · Base de Datos Compartida	9
Nuevas tablas de Supabase	9
Sincronización en tiempo real	9
7 · Fases de Desarrollo	10
8 · Sugerencias de Desarrollo y Expansión	11

Mejoras técnicas internas	11
Expansiones de capacidad	12
Integraciones externas	12
Visión a largo plazo	13

Visión del Proyecto

PROPÓSITO

Elfie-PRTS Desktop no reemplaza la versión web — la extiende. Mientras PRTS web es accesible desde cualquier dispositivo, Elfie Desktop vive permanentemente en la máquina de Sylft, tiene acceso nativo al sistema operativo, procesa IA localmente y funciona como asistente ambiental de escritorio con personalidad propia.

Estado actual de PRTS Web — Qué ya existe

ÁREA	ESTADO	DETALLE
Frontend	COMPLETO	HTML/JS vanilla, 10 módulos funcionales, PWA
Base de datos	COMPLETO	Supabase Postgres, 11 migraciones, RLS completo, 27+ tablas
Capa IA (Anthropic)	COMPLETO	4 Edge Functions: briefing, inbox, comandos, insights
Voz web	COMPLETO	webkitSpeechRecognition + speechSynthesis (es-MX)
Wake word "Dalia"	COMPLETO	Escucha continua en desktop Chromium
Google Calendar	COMPLETO	Recordatorios → notificación al celular
Alumnos / Contenido	PENDIENTE	Solo nodos en el mapa, sin HTML ni tablas

Qué cambia y añade Elfie Desktop

CAPACIDAD	PRTS WEB	ELFIE DESKTOP
Control del SO	Solo deep links (protocol handlers)	Nativo: procesos, ventanas, volumen, archivos
STT	Google (cloud, privacidad limitada)	Whisper.cpp local, sin Google, funciona offline
Wake word	webkitSpeechRecognition continuo	Picovoice Porcupine on-device (real nivel C)
TTS / Voz de Elfie	speechSynthesis del navegador	Kokoro / XTTS v2 local, clonación de voz

CAPACIDAD	PRTS WEB	ELFIE DESKTOP
LLM	Solo Anthropic API (costo por llamada)	Ollama local (RTX 5060) + Anthropic fallback
Memoria contextual	Sin memoria entre sesiones	RAG local (ChromaDB / LanceDB)
Disponibilidad	Necesita navegador abierto	Tray icon, siempre activo, auto-start
Monitoreo	Sin métricas del sistema	CPU/GPU/RAM/temperatura en tiempo real
Personalidad	Respuestas funcionales	Perfiles configurables, nombre Elfie, tono propio

02

Decisiones de Arquitectura

FRAMEWORK DE ESCRITORIO

Tauri 2.x (Rust + WebView)

Elegido sobre Electron por ser 15× más liviano (~8 MB vs ~150 MB), backend en Rust para operaciones nativas de alto rendimiento, y porque el frontend puede reutilizar **exactamente** el código HTML/JS vanilla de PRTS. La identidad visual migra sin reescritura.



Hardware disponible y uso estratégico

GPU

RTX 5060 · 8 GB
VRAM

LLM local (hasta 7B Q4),
Whisper large-v3-turbo, XTTS
v2 · Deja ~3GB libres para el
sistema

RAM

16 GB DDR4

Elfie + Ollama + sistema:
presupuesto ~6 GB para la app;
10 GB para el SO y otras
aplicaciones

CPU

**Ryzen 5 5600G · 6C/
12T**

Tauri Rust backend + Piper TTS
(CPU) como fallback; Porcupine
wake word en CPU dedicado

GESTIÓN DE VRAM (RTX 5060 · 8 GB)

Presupuesto de VRAM sugerido: LLM Mistral 7B Q4 ~4.5 GB · Whisper large-v3-turbo ~1.5 GB · XTTS v2 ~2.5 GB. No corren todos simultáneamente. El router prioriza: si hay comando en curso, el LLM toma ~5 GB; el STT siempre activo (~1.5 GB); el TTS solo se

carga al responder (~2.5 GB durante ~2 s). Total peak: ~5 GB. Pico máximo ocasional: ~7.5 GB.

03

Módulos del Sistema

Control del Sistema Operativo

Implementado en Rust (`system_control.rs`), expuesto al frontend vía comandos Tauri IPC.

COMANDO	DESCRIPCIÓN	EJEMPLO DE USO POR VOZ
<code>open_app</code>	Lanza ejecutables por nombre o ruta. Alias configurables en <code>elfie_config</code>	"Elfie, abre Discord"
<code>kill_process</code>	Termina procesos por nombre o PID	"Elfie, cierra Steam"
<code>list_windows</code>	Lista ventanas abiertas para cambio de foco	"Elfie, ¿qué tengo abierto?"
<code>set_volume</code>	Volumen maestro del sistema (0-100)	"Elfie, sube el volumen a 60"
<code>mute_toggle</code>	Silenciar/activar audio	"Elfie, silencia"
<code>clipboard_read</code>	Lee el portapapeles actual	"Elfie, ¿qué tengo copiado?"
<code>clipboard_write</code>	Escribe en el portapapeles	Usado internamente por acciones
<code>open_folder</code>	Abre carpeta en el Explorador de Windows	"Elfie, abre mis proyectos"
<code>screenshot</code>	Captura de pantalla → guarda en disco o clipboard	"Elfie, toma una captura"
<code>power_action</code>	Apagar / reiniciar / suspender	"Elfie, suspende la computadora"
<code>run_routine</code>	Secuencia de acciones predefinida	"Elfie, modo trabajo"

Monitor de Recursos en Tiempo Real

Usa la crate `sysinfo` (CPU/RAM) y `nvml-wrapper` (GPU NVIDIA) en Rust. Los datos se emiten cada 3 segundos al frontend vía eventos Tauri.

MÉTRICAS DISPONIBLES

- CPU: porcentaje de uso por núcleo + total · frecuencia actual · temperatura

- GPU RTX 5060: % de uso · VRAM usada/total · temperatura · potencia consumida · velocidad del ventilador
- RAM: usada / disponible / total (16 GB)
- Disco: espacio libre por partición
- Red: Mbps de bajada y subida en tiempo real
- Procesos: top-10 por consumo de CPU y de RAM

Personalidad de Elfie

La personalidad es configurable y se persiste en la tabla `elfie_config` de Supabase, sincronizada entre escritorio y web.

PERFIL	TONO	TTS	VERBOSIDAD
Profesional (default)	Directa, eficiente, sin rodeos	Voz neutra, velocidad 1.0	Concisa
Casual	Informal, puede hacer comentarios breves	Voz más cálida, velocidad 1.05	Media
Silenciosa	Mínima: solo confirmaciones de 1-2 palabras	TTS desactivado, solo texto	Mínima
Técnica	Datos concretos, métricas, sin suavizar	Voz neutra, velocidad 0.95	Detallada
Nocturna	Más suave, volumen reducido automáticamente	Voz suave, velocidad 0.9	Mínima

Dashboard Sincronizado

SINCRONIZACIÓN BIDIRECCIONAL VIA SUPABASE REALTIME

Elfie Desktop se suscribe a los canales Realtime de Supabase (`tasks` , `daily_briefings` , `reminders` , `inbox_entries`). Cualquier cambio desde la web se refleja en el escritorio en <1 segundo, y viceversa. No hay polling ni refrescos manuales.

04

Capa de Voz

STT Local — Whisper.cpp

Reemplaza `webkitSpeechRecognition` (dependía de Google, requería navegador abierto). Con Whisper.cpp:

- El audio **nunca sale de la computadora** — privacidad total
- Funciona **sin internet**
- Precisión superior en **español mexicano**, especialmente con vocabulario técnico/gym/dieta
- Modelo `large-v3-turbo`: 1.6 GB VRAM, transcripción en ~300 ms para frases cortas
- Integración en Tauri vía crate `whisper-rs` o sidecar Python

TTS Local (sin costo, aprovecha RTX 5060)

Kokoro TTS RECOMENDADO LOCAL · 82M PARAMS · ONNX <table><tr><td>Calidad</td><td>Excelente</td></tr><tr><td>Latencia</td><td><100 ms (GPU)</td></tr><tr><td>VRAM</td><td>~0.5 GB</td></tr><tr><td>es-MX</td><td>Sí</td></tr><tr><td>Costo</td><td>Gratuito</td></tr></table>	Calidad	Excelente	Latencia	<100 ms (GPU)	VRAM	~0.5 GB	es-MX	Sí	Costo	Gratuito	XTTS v2 (Coqui) LOCAL · CLONACIÓN DE VOZ <table><tr><td>Calidad</td><td>Muy alta</td></tr><tr><td>Latencia</td><td>~500 ms (GPU)</td></tr><tr><td>VRAM</td><td>~2.5 GB</td></tr><tr><td>Clonación</td><td>Sí, 5 s de referencia</td></tr><tr><td>Costo</td><td>Gratuito</td></tr></table>	Calidad	Muy alta	Latencia	~500 ms (GPU)	VRAM	~2.5 GB	Clonación	Sí, 5 s de referencia	Costo	Gratuito
Calidad	Excelente																				
Latencia	<100 ms (GPU)																				
VRAM	~0.5 GB																				
es-MX	Sí																				
Costo	Gratuito																				
Calidad	Muy alta																				
Latencia	~500 ms (GPU)																				
VRAM	~2.5 GB																				
Clonación	Sí, 5 s de referencia																				
Costo	Gratuito																				
Piper TTS LOCAL · CPU · ULTRARRÁPIDO <table><tr><td>Calidad</td><td>Buena</td></tr><tr><td>Latencia</td><td><50 ms (CPU)</td></tr><tr><td>VRAM</td><td>0 (solo CPU)</td></tr><tr><td>es-MX</td><td>Modelos disponibles</td></tr><tr><td>Costo</td><td>Gratuito</td></tr></table>	Calidad	Buena	Latencia	<50 ms (CPU)	VRAM	0 (solo CPU)	es-MX	Modelos disponibles	Costo	Gratuito	Fish Speech LOCAL · ESTADO DEL ARTE <table><tr><td>Calidad</td><td>Estado del arte</td></tr><tr><td>Latencia</td><td>~200 ms (GPU)</td></tr><tr><td>VRAM</td><td>~4 GB</td></tr><tr><td>Clonación</td><td>Sí</td></tr><tr><td>Costo</td><td>Gratuito</td></tr></table>	Calidad	Estado del arte	Latencia	~200 ms (GPU)	VRAM	~4 GB	Clonación	Sí	Costo	Gratuito
Calidad	Buena																				
Latencia	<50 ms (CPU)																				
VRAM	0 (solo CPU)																				
es-MX	Modelos disponibles																				
Costo	Gratuito																				
Calidad	Estado del arte																				
Latencia	~200 ms (GPU)																				
VRAM	~4 GB																				
Clonación	Sí																				
Costo	Gratuito																				

TTS en la Nube (costo variable, máxima calidad)

Microsoft Edge TTS GRATUITO CLOUD · EDGE-TTS · SIN API KEY <table><tr><td>Voz es-MX</td><td>es-MX-DaliaNeural ★</td></tr><tr><td>Latencia</td><td>~200 ms</td></tr><tr><td>Costo</td><td>Gratuito (500k chars/mes)</td></tr></table>	Voz es-MX	es-MX-DaliaNeural ★	Latencia	~200 ms	Costo	Gratuito (500k chars/mes)	ElevenLabs CLOUD · MÁXIMA CALIDAD <table><tr><td>Calidad</td><td>La mejor del mercado</td></tr><tr><td>Latencia</td><td>~300 ms</td></tr><tr><td>Clonación</td><td>Sí (plan Creador)</td></tr></table>	Calidad	La mejor del mercado	Latencia	~300 ms	Clonación	Sí (plan Creador)
Voz es-MX	es-MX-DaliaNeural ★												
Latencia	~200 ms												
Costo	Gratuito (500k chars/mes)												
Calidad	La mejor del mercado												
Latencia	~300 ms												
Clonación	Sí (plan Creador)												

NotaLa voz se llama "Dalia" — encaja perfecto con la wake word

Costo\$5-22/mes según uso

OpenAI TTS

CLOUD · VOCES NATURALES

Voces

alloy, echo, nova, shimmer...

Latencia

~500 ms

Costo

\$15/1M caracteres

es-MX

Sí (multilingüe)

Azure Cognitive TTS

CLOUD · MICROSOFT

Voces es-MX

es-MX-DaliaNeural (Neural)

Costo

~\$4/1M chars (Neural)

SSML

Sí — control de tono, pausa, énfasis

Streaming

Sí — respuesta progresiva

Stack de Voz Recomendado

ESTRATEGIA EN CAPAS (PRIORIDAD DESCENDENTE)

1 · Kokoro TTS (local, GPU) — respuestas cotidianas, <100 ms, sin costo

2 · Piper TTS (local, CPU) — fallback si la GPU está saturada, <50 ms en CPU

3 · edge-tts es-MX-DaliaNeural (cloud gratuita) — si los modelos locales no están cargados

4 · ElevenLabs / XTTS v2 (opcional) — voz clonada para momentos de mayor presencia

La elección de motor se persiste en `elfie_config.voice_engine` y es intercambiable en tiempo de ejecución desde el panel de configuración.

05

IA Local (RTX 5060 · 8 GB VRAM)

HERRAMIENTA: OLLAMA

Ollama actúa como runtime local de LLMs. Expone una API REST compatible con OpenAI en `localhost:11434`, por lo que el código del router existente (que ya llama a un endpoint REST) cambia mínimamente para soportar el modo local.

Modelos para RTX 5060 (8 GB VRAM)

MODELO	CUANTIZACIÓN	VRAM	VELOCIDAD	CALIDAD	USO IDEAL EN ELFIE
Phi-3.5 Mini	Q4_K_M	~2.5 GB	~120 tok/s	Muy buena	Router local, comandos rápidos, respuestas simples
Llama 3.2 3B	Q8	~3.5 GB	~80 tok/s	Buena	Respuestas de asistente, conversación casual
Mistral 7B v0.3	Q4_K_M	~4.5 GB	~50 tok/s	Excelente	Inbox, clasificación, análisis de texto
Llama 3.1 8B	Q4_K_M	~5.0 GB	~40 tok/s	Excelente	Conversación profunda, preguntas complejas (intent ask)
Qwen2.5 7B	Q4_K_M	~4.5 GB	~50 tok/s	Excelente	Mejor en español, razonamiento matemático

Estrategia Híbrida Local + Nube

ROUTER DE IA – Decisión de motor Comando recibido (voz o texto) | ▼ ¿Patrón local sin LLM? → SÍ → Resolver en cliente (0 ms, 0 costo) | NO (hora, fecha, saludo, navegar...) ▼ ¿Ollama disponible? → SÍ → Phi-3.5 Mini / Mistral 7B local | NO (~300-800 ms, \$0) ▼ ¿Anthropic API disponible? → SÍ → Claude Haiku 4.5 | NO (~600 ms, ~\$0.001/llamada) ▼ Modo degradado: respuesta determinista / manual (sin IA)

Casos de uso por motor

INTENCIÓN	MOTOR	POR QUÉ
Hora, fecha, saludo	Local sin LLM	Cero latencia

INTENCIÓN	MOTOR	POR QUÉ
Abrir app, navegar	Local sin LLM	Whitelist estática
Clima (Open-Meteo)	Local sin LLM	API gratuita, sin LLM
Crear tarea / recordatorio	Ollama local	Phi-3.5 Min — privacidad
Clasificar inbox	Ollama local	Mistral 7B — sin costo recurrente
Pregunta general (ask)	Ollama local	Llama 3.1 8B — offline
Briefing diario (complejo)	Anthropic API	Contexto amplio, mayor calidad
Insights de 4 semanas	Anthropic API	Análisis profundo, contexto largo
Cualquier cosa en modo offline	Ollama local	Fallback completo

AHORRO ESTIMADO CON IA LOCAL

Actualmente, PRTS web gasta Anthropic API en clasificación de inbox (~5 llamadas/día) + comandos de voz (~10/día) + briefing (1/día). Con Elfie Desktop, las clasificaciones e intents simples van a Ollama local. **Estimado: reducción del 70–80% del gasto mensual en API de IA.** Solo briefings e insights quedan en Claude API por calidad.

06

Base de Datos Compartida

La base de datos de Supabase existente es la fuente de verdad única para ambas apps. Elfie Desktop se conecta con las mismas credenciales (`SUPABASE_URL` + `SUPABASE_ANON_KEY`), el mismo RLS, y la misma API cliente JavaScript reutilizada desde PRTS web.

Nuevas Tablas (migraciones 0012–0015)

```
-- Migración 0012: Configuración de Elfie y perfiles de voz
CREATE TABLE elfie_config (
  user_id uuid DEFAULT auth.uid() PRIMARY KEY,
  personality text DEFAULT 'profesional',
  voice_engine text DEFAULT 'kokoro', -- kokoro|piper|xtts|edge|elevenlabs
  voice_name text DEFAULT 'es-MX-DaliaNeural',
  voice_speed float DEFAULT 1.0,
  voice_pitch float DEFAULT 0.0,
  verbosity text DEFAULT 'concisa', -- minima|concisa|media|detallada
  tts_enabled bool DEFAULT true,
  local_llm_model text DEFAULT 'phi3.5',
  theme text DEFAULT 'dark',
  updated_at timestampz DEFAULT now() );
-- Migración 0013: Perfiles de voz personalizados (clonación)
CREATE TABLE voice_profiles (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id uuid DEFAULT auth.uid(),
  name text NOT NULL,
  engine text NOT NULL, -- xtts|fish|elevenlabs
  reference_audio_url text, -- audio de referencia para clonar
  config jsonb, -- parámetros extra del motor
  is_active bool DEFAULT false,
  created_at timestampz DEFAULT now() );
-- Migración 0014: Log de acciones del sistema operativo
CREATE TABLE system_actions (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id uuid DEFAULT auth.uid(),
  action_type text NOT NULL, -- open_app|volume|file|routine...
  payload jsonb,
  executed_at timestampz DEFAULT now() );
-- Migración 0015: Métricas del sistema (historial + dashboard)
CREATE TABLE system_metrics (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id uuid DEFAULT auth.uid(),
  cpu_percent float,
  ram_used_gb float,
  gpu_percent float,
  gpu_vram_used_gb float,
  gpu_temp_c float,
  gpu_power_w float,
  recorded_at timestampz DEFAULT now() );
```

Sincronización en Tiempo Real

CANAL SUPABASE REALTIME	DIRECCIÓN	QUÉ SINCRONIZA
tasks	Web ↔ Desktop	Nueva tarea en web → aparece en Elfie en <1 s
daily_briefings	Web → Desktop	Briefing pre-generado disponible → Elfie lo muestra
reminders	Web ↔ Desktop	Recordatorios creados en cualquier cliente
inbox_entries	Web ↔ Desktop	Capturas procesadas en web o desktop
ai_insights	Web → Desktop	Insights generados → Elfie notifica nativamente

Fases de Desarrollo

F0 Setup Base

~1 sem

- Inicializar proyecto Tauri 2.x
- Cargar frontend PRTS existente en WebView
- Conexión a Supabase (mismas credenciales)
- Autenticación reutilizada de PRTS
- Verificar que los módulos actuales funcionan en el WebView

F1 Core Desktop

~2 sem

- Tray icon con menú contextual
- Auto-start en Windows (registro)
- Notificaciones nativas de Windows
- Supabase Realtime (sincronización live)
- Dashboard Elfie en la tray: briefing + tareas del día

F2 Control del Sistema

~2 sem

- Launcher de aplicaciones (`open_app`)
- Control de volumen del sistema
- Monitor CPU/GPU/RAM en tiempo real
- Gestión básica de archivos y carpetas
- Portapapeles (leer/escribir)
- Rutinas de inicio configurables

F3 Voz Local

~2 sem

- Whisper.cpp como STT (sin Google)
- Kokoro TTS / Piper TTS integrado
- Wake word con Picovoice Porcupine
- Flujo completo: wake → Whisper → router → TTS
- Indicador visual de estado (escucha/habla)

F4 IA Local

~2-3 sem

- Instalar Ollama, descargar modelos base
- Router híbrido: local → Anthropic fallback
- Personalidad de Elfie configurable
- Perfiles de voz (múltiples motores)
- Panel de configuración de personalidad en UI
- Memoria contextual con ChromaDB (RAG)

F5 Módulos Nuevos

~2 sem

- Panel de monitor de sistema completo
- Historial de `system_metrics` en gráficas
- Routine Engine visual (crear secuencias)
- Módulo Alumnos (tabla + UI pendiente)
- Módulo Contenido (planificador de clases/streams)

F6 Clonación de Voz

~1 sem

- XTTS v2 integrado como opción de motor
- Interfaz para subir audio de referencia
- Almacenamiento del perfil en `voice_profiles`
- Previsualización antes de activar

F7 Distribución

~1 sem

- Installer .msi / .exe para Windows
- Auto-update con Tauri updater
- Firma del ejecutable (opcional)
- Script de configuración inicial (Ollama + modelos)

TIEMPO TOTAL ESTIMADO

~13–15 semanas de desarrollo a ritmo de proyecto personal. Las Fases 0–3 entregan la app funcional básica en ~7 semanas. Las Fases 4–7 añaden la diferenciación real (IA local, personalidad, clonación de voz). Es recomendable hacer uso real después de cada fase antes de avanzar — misma filosofía de PRTS.

Sugerencias de Desarrollo y Expansión

Mejoras Técnicas Internas

RAG local con ChromaDB / LanceDB

Elfie recuerda conversaciones, notas y contexto histórico. Al preguntar algo, recupera información relevante de las últimas semanas antes de llamar al LLM. Sin cloud, sin costo.

Plugin tauri-plugin-global-shortcut

Atajos globales de teclado (funcionan aunque Elfie esté en segundo plano). Ej: Ctrl+Space abre el input de captura; Alt+E activa escucha de voz.

Offline-first completo

Elfie funciona sin internet: Ollama local, Whisper local, Kokoro local. Al reconectar, sincroniza con Supabase las acciones acumuladas (cola de operaciones pendientes).

Auto-update silencioso

Tauri updater apuntando a GitHub Releases. Elfie se actualiza en segundo plano y notifica al reiniciar. Sin intervención manual.

Evals para LLM local

Extender el sistema de evals existente (ai/evals/) para correr contra Ollama local antes de cambiar de modelo. Mide precisión de clasificación en inbox y comandos.

NVML para métricas GPU reales

La crate nvml-wrapper de Rust da acceso a VRAM, temperatura, potencia y uso de la RTX 5060 en tiempo real. Mucho más preciso que sysinfo genérico.

Sidecar pattern para Python

Empaquetar Whisper y TTS como procesos sidecar de Tauri. El usuario no instala Python; el instalador incluye el entorno embebido. Setup de un solo clic.

Panel de costos de IA integrado

Mostrar en la UI cuánto se gastó en Anthropic API este mes (ya existe ai_calls en Supabase) vs. cuántas llamadas se resolvieron localmente. Decisión informada sobre qué mover a local.

Expansiones de Capacidad

Routine Engine Visual

"Modo trabajo" = abrir VS Code + Discord + Spotify playlist foco + silenciar notificaciones + poner volumen a 40. Elfie ejecuta la secuencia con un comando. Rutinas guardadas en Supabase.

Time Tracker automático

Detectar qué aplicación tiene el foco (GetForegroundWindow en Windows) y registrar tiempo activo por app. Elfie analiza patrones: "pasaste 3 h en VS Code hoy". Sin datos externos.

Módulo Alumnos completo

Para Wolves robótica y LevelUp idiomas: registro de asistencia, progreso por alumno, calendario de sesiones, notas de clase. Elfie

Modo Docente con IA

Al dar clase, Elfie asiste: traducciones instantáneas (LLM local), definiciones, ejemplos generados en tiempo real. "Elfie, ¿cómo explico recursión en inglés?"

puede sugerir qué repasar en la próxima sesión.

Integración con Obsidian

Los Apuntes de PRTS sincronizan automáticamente con un vault de Obsidian local. Escribe en PRTS, abre en Obsidian con sus plugins de backlinks y graph view.

Screenshot + descripción IA

Elfie puede capturar la pantalla y que el LLM multimodal la describa o extraiga texto/código. Útil para agregar contexto visual al inbox sin teclear.

Monitor de progreso TecNM

Módulo Escuela: fechas de exámenes, proyectos, calificaciones. Elfie alerta cuando se acerca un parcial. Integra con los Apuntes existentes por materia.

Módulo Contenido / Stream

Planificación de contenido para YouTube/ Twitch/Redes. Ideas → guiones → calendario de publicación. Elfie sugiere temas basados en tendencias (intent ask con búsqueda web opcional).

Integraciones Externas y Visión a Largo Plazo

08

Integraciones con Apps ya en uso

Integración profunda con Discord

Via Discord RPC + webhook: Elfie puede cambiar el estado "Ahora jugando", enviar mensajes a canales definidos, o alertar cuando llega un mensaje de alguien específico. "Elfie, dile a #wolves que llego en 10 min".

Spotify Web API real

Cambiar de los presets estáticos actuales a la API real de Spotify: reproducir cualquier playlist/canción por nombre, ver la canción actual, control de reproducción por voz. OAuth de usuario (ya hay client_id en config).

OBS Studio Integration

Via OBS WebSocket: cambiar escena, iniciar/detener grabación/stream, activar filtros. "Elfie, empieza la grabación", "Elfie, cambia a escena de juego". Útil si hace contenido.

VS Code Extension

Elfie como extension de VS Code: crear tareas o apuntes desde el editor, "Elfie, agrega a pendientes revisar este bug", abrir archivos de proyectos PRTS por voz.

WhatsApp via wa-automate / Baileys

Leer el último mensaje de un contacto, responder mensajes predefinidos, o notificar a contactos. Requiere un número/sesión dedicada. "Elfie, ¿tengo mensajes nuevos de mamá?"

Google Calendar bidireccional

Actualmente PRTS solo escribe a Calendar. Con Elfie Desktop: leer eventos del día desde Calendar y mostrarlos en el briefing, sincronizar ambas vías sin abrir el navegador.

Visión a Largo Plazo — Elfie como Plataforma

ELFIE COMO SISTEMA NERVIOSO PERSONAL

La visión final de Elfie-PRTS Desktop no es solo una app de escritorio mejorada — es un **sistema nervioso personal**: toda la información de Sylft (estudios, gym, finanzas, trabajo, proyectos, alumnos) pasa por un único punto de control que aprende, conecta y facilita. Elfie actúa en segundo plano: agenda, advierte, recuerda, ejecuta y reporta.

Elfie Mobile (React Native / Flutter)

App móvil nativa que comparte la misma base de datos Supabase. Para los días sin computadora: captura por voz en el celular, Elfie clasifica y procesa al llegar al escritorio.

Fine-tuning de LLM con tus datos

Con ~6 meses de datos en Supabase (tareas, inbox, gym, dieta), hacer fine-tuning de un modelo pequeño (Phi-3.5, Qwen2.5) para que entienda el vocabulario y contexto personal de Sylft sin prompts largos.

Companion nativo (Electron companion)

Proceso de sistema sin ventana que hace el nivel C real: funciona con el navegador cerrado, inicia con Windows, escucha la wake word permanentemente usando Picovoice (WASM on-device).

API pública de Elfie

Exponer una API local (localhost:XXXX) para que otras herramientas (scripts, VS Code, Alfred/PowerToys) puedan hablarle a Elfie directamente sin abrir la app.